

Gradient Boosting Algorithm

Input: training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$, number of iterations M .

Algorithm:

1. Initialize model with a constant value:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma).$$

2. For $m = 1$ to M :

1. Compute so-called *pseudo-residuals*:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n.$$

2. Fit a base learner (or weak learner, e.g. tree) closed under scaling $h_m(x)$ to pseudo-residuals, i.e. train it using the training set $\{(x_i, r_{im})\}_{i=1}^n$.

3. Compute multiplier γ_m by solving the following

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)).$$

4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

3. Output $F_M(x)$.

Gradient Boosting (GB):

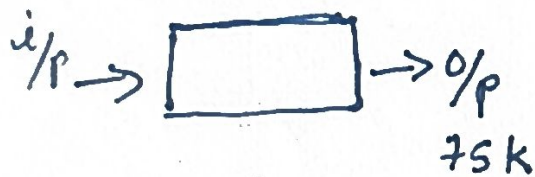
- ↳ It is supervised machine learning ensemble technique that combines multiple weak learners to create a final model.
- ↳ It sequentially combine multiple weak learners
- ↳ Typically decision tree to create single strong predictive model.
- ↳ GB involves training each model to correct the errors of previous models.
- ↳ Each new model is trained to minimize the loss function (mean squared error or cross entropy of previous model using gradient descent).
- ↳ In each iteration the algorithm computes the gradient of the loss function with respect to predictions then trains a new weak model to minimize the gradient.
- ↳ Prediction of the new model are then added to the ensemble (all models prediction) and the process is repeated until a stopping criteria is met.
- ↳ Thus, GB based on boosting in a functional space, where target is pseudo-residuals
- ↳ A decision tree is the weak learner, the resulting algorithm is called gradient-boosted trees.

Gradient Boost explain steps with example.

Dataset:

x_1	x_2	y	$(y - \hat{y})$
Experience	Degree	Salary	R_1
2	B.E	50K	-25K
3	M. Tech	70K	-5K
5	M. Tech + Certi fication	80K	5K
6	PhD	100K	25K
	Avg:	75K	

Step1: create a base model



Step2: Compute residual error

$$75 - 50K = -25$$

$$75 - 70K = -5K$$

$$75 - 80K = 5K$$

$$75 - 100K = 25K$$

Step3: Construct decision tree

Gradient Boosting Example

Ex

iq	CGPA	lpa
90	8	3
100	7	4
110	6	8
120	9	6
80	5	3

First model:

Compute initialize the model with average value

$$\therefore \text{pred 1} : \frac{3 + 4 + 8 + 6 + 3}{5} = 4.8$$

$$\therefore$$

iq	CGPA	lpa	pred 1
90	8	3	4.8
100	7	4	4.8
110	6	8	4.8
120	9	6	4.8
80	5	3	4.8

Second model:

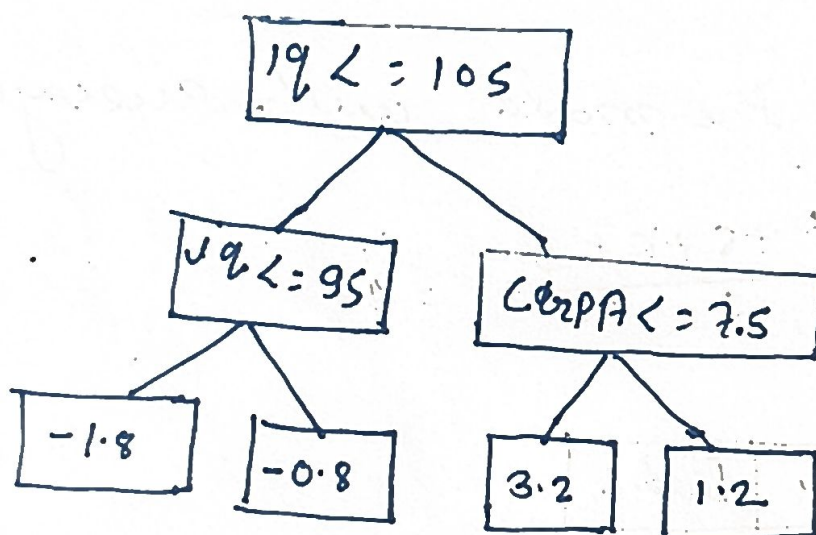
Compute residual/error

$$\begin{array}{lcl}
 4.8 - 3 = & 3 - 4.8 = & -1.8 \\
 4.8 - 4 = & 4 - 4.8 = & -0.8 \\
 4.8 - 8 = & 8 - 4.8 = & 3.2 \\
 4.8 - 6 = & 6 - 4.8 = & 1.2 \\
 4.8 - 3 = & 3 - 4.8 = & -1.8
 \end{array}$$

Now compute / create tree
model 2 is decision tree

IQ	CGPA	Salary	pred1	res1
90	8	3	4.8	-1.8
100	7	4	4.8	-0.8
110	6	8	4.8	3.2
120	9	6	4.8	1.2
80	5	3	4.8	-1.8

Create decision tree.



Now prediction on basis of above decision tree.
So.

IQ	CGPA	Salary	pred1	res1	pred2
90	8	3	4.8	-1.8	-1.8
100	7	4	4.8	-0.8	-0.8
110	6	8	4.8	3.2	3.2
120	9	6	4.8	1.2	1.2
80	5	3	4.8	-1.8	-1.8

Same prediction as residual.

8 It seems like overfitting.

If we consider GB have only two model. then prediction will be:

$$\text{pred} = m_1 + m_2$$

Suppose we consider an example

here iq = 90 & CUPH = 8 then salary?

$$\text{pred} = m_1 + m_2$$

$$= 4.8 - 1.8 = 3$$

and we see ^{that} actual salary is 3.

Similarly compute for model prediction for other examples in table.

$$\text{pred} = m_1 + m_2$$

$$= 4.8 - 0.8 = 4$$

Here we see that prediction using GB is same as actual which shows overfitting of model.

therefore we add the learning rate.

$$\therefore \text{pred} = m_1 + \cancel{m_2} \times \text{learning rate} = 0.1$$

Now prediction operation perform for student 1.

$$\text{pred} = m_1 + 0.1 \times (4.8)$$

$$= 4.8 - 0.18 = 4.7.$$

here predicted output is different from actual output. For correct prediction we add more model in final prediction.